

LangChain

LangChain is a popular framework that helps developers create AI applications by chaining prompts, memory, and various tools together. It simplifies the process of integrating AI into different workflows.

Key features:

- Modular design for creating and managing AI workflows.
- Built-in memory to enable stateful conversations.
- Compatibility with multiple AI models and APIs.

Use case: A chatbot for a bank that retrieves customer details and provides insights based on transaction history.

LangGraph

LangGraph extends LangChain by introducing a graph-based approach to managing workflows. This framework is useful for applications that require step-by-step task execution.

Key features:

- Graph-based workflow management for AI agents.
- Handles dependencies between different tasks efficiently.
- Supports complex multi-step processes.

Use case: A medical diagnosis AI that retrieves patient history, analyses symptoms, and suggests treatments in a structured workflow.

Microsoft Semantic Kernel

Microsoft Semantic Kernel focuses on making AI systems understand user intent and context better. It is particularly useful for AI applications that require deep semantic reasoning.

Key features:

- Context-aware decision-making for AI applications.
- Pre-built integrations with enterprise tools and systems.
- Enhanced semantic understanding for improved accuracy.

Use case: An AI-powered IT helpdesk that understands user queries and suggests the best solutions based on past interactions.

CrewAI

CrewAI is designed for multi-agent collaboration. It allows AI agents to take on different roles and work as a team to complete tasks efficiently.

Key features:

- Role-based AI agents with specialised tasks.
- Customisable tools and workflows for different use cases.
- Task execution can be sequential or hierarchical.

Use case: A logistics management AI system where

agents plan delivery routes and optimise warehouse operations collaboratively.

PhiData Agentic Framework

The PhiData Agentic Framework enhances AI assistants by integrating memory, knowledge, and tools to handle complex tasks.

Key features:

- AI memory to retain past interactions.
- Access to multiple data sources, including databases and PDFs.
- Built-in tools for web search, document summarisation, and query execution.

Use case: A business AI assistant that retrieves reports, answers queries, and automates document processing.

Building agentic AI: A hands-on guide for developers

The goal of this system is to develop an AI-driven multi-agent service that automatically classifies user queries into different categories and routes them to specialised AI agents for appropriate responses. Specifically, the system categorises queries into:

- Weather-related questions (handled by the Weather Service Agent)
- News-related enquiries (handled by the News Service Agent)
- Joke requests (handled by the Joke Service Agent)

Importing dependencies

We begin by importing dependencies as follows.

- *langgraph*: Used for managing workflows and state transitions.
- *time*: Used for measuring performance.
- *HuggingFaceHub* and *SystemMessage* from *langchain*: To interact with the Hugging Face model hub and work with prompts.
- *StateGraph* from *langgraph*: To define a state machine that orchestrates the flow of different agents.
- *BaseModel* from *pydantic*: For defining structured data models (e.g., *ExpertSystemState*).
- *os*: For handling environment variables, specifically to set the Hugging Face API key.

Importing Dependencies

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
import langgraph
import time
from langchain.llms import HuggingFaceHub
```